

@aiwithcasti

How Award-Winning Websites Are Really Built

3D scenes · Scroll-driven storytelling · Cinematic intros
The real tools and tricks behind Awwwards-tier sites

No fluff. Real tools, real code, real AI prompts — the actual stack studios use, not recycled buzzwords.

WHAT'S INSIDE

Table of Contents

- | | |
|----|--|
| 01 | The Real Stack
What award sites are actually built with |
| 02 | 3D Scenes That Don't Lag
Three.js, Spline, and when to use which |
| 03 | The Image-Sequence Trick
How Apple-style scroll-zoom actually works |
| 04 | Scroll Storytelling
GSAP ScrollTrigger + Lenis, pinning, reveals |
| 05 | Cinematic Intro Loaders
Preloaders that set the tone before content loads |
| 06 | Color & Motion Direction
Why award sites all feel cohesive |
| 07 | AI Shortcuts For Every Step
Prompts that replace hours of manual setup |
| 08 | Performance Reality Check
Where 3D sites actually die, and how to avoid it |
-

The Real Stack

Award sites aren't built with one secret tool. They combine a handful of proven libraries the same way every time. Here's what's actually running under the hood — verified, not guessed.

Three.js

The core 3D rendering engine for the web (WebGL). Used for particles, geometry, shaders, real-time scenes.

GSAP + ScrollTrigger

The animation engine. Drives every scroll-pinned section, reveal, and timeline sequence.

Lenis

Smooth-scroll wrapper. Makes scrolling feel buttery and gives GSAP a clean scroll value to hook into.

Spline

Browser-based 3D design tool. Build & light 3D scenes visually, export directly to the web — no Three.js code required.

Theatre.js

Visual animation director for Three.js scenes. Lets you keyframe 3D objects in a GUI instead of hardcoding values.

Framer / Webflow

No-code builders with native scroll & hover animation support — used for the layout shell around the 3D/heavy bits.

■ PRO TIP

You don't need to master all six. Pick GSAP + Lenis for motion, then ADD Three.js or Spline only if the brief truly needs 3D.

3D Scenes That Don't Lag

Real-time 3D is the single biggest reason award-site clones break on mobile. Here's how to choose the right approach for the result you actually need.

Three.js — when you write code

Use it when the 3D needs to react dynamically to data, user input, or be fully custom (particles, shaders, generative geometry). It is the most powerful option and the heaviest to build and maintain.

JS — Minimal Three.js scene setup

```
import * as THREE from 'three';

const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(
  75, innerWidth / innerHeight, 0.1, 1000);
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(innerWidth, innerHeight);
document.body.appendChild(renderer.domElement);

const geometry = new THREE.IcosahedronGeometry(1, 0);
const material = new THREE.MeshStandardMaterial({ color: 0x0d9488 });
const mesh = new THREE.Mesh(geometry, material);
scene.add(mesh);

function animate() {
  mesh.rotation.y += 0.01;
  renderer.render(scene, camera);
  requestAnimationFrame(animate);
}
animate();
```

Spline — when you design visually

Use it when the scene is mostly static or lightly interactive (rotate on hover, react to scroll position) and you want to design lighting and materials by eye instead of math. Export as a React component or embed code directly.

JSX — Embedding a Spline scene in React

```
import Spline from '@splinetool/react-spline';

export default function Hero() {
  return (
    <Spline scene='https://prod.spline.design/your-scene/scene.splinecode' />
  );
}
```

■ **PRO TIP**

Rule of thumb: if the brief says 'rotating 3D object in the hero,' use Spline. If it says 'particles that react to the cursor,' use Three.js.

The Image-Sequence Scroll-Zoom Trick

This is the real secret behind Apple's product pages and most '3D scroll zoom' effects on award sites. It is NOT live 3D rendering — it's a sequence of pre-rendered images played back on scroll.

How it actually works

1. A 3D model (made in Blender, Spline, or a 3D render tool) is rendered as a sequence of 60-150 still frames, rotating or zooming through the animation.
2. Each frame is exported as an optimized image (WebP) and numbered sequentially.
3. GSAP ScrollTrigger maps scroll position to a frame number and draws that frame onto a element.
4. Because it's just swapping 2D images, it runs at 60fps on any device — including phones that would choke on real-time 3D.

JS — Canvas image-sequence scroll scrubber (core pattern)

```
const canvas = document.querySelector('canvas');
const ctx = canvas.getContext('2d');
const frameCount = 120;
const images = [];

for (let i = 0; i < frameCount; i++) {
  const img = new Image();
  img.src = `/frames/frame_${String(i).padStart(4, '0')}.webp`;
  images.push(img);
}

function render(index) {
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  ctx.drawImage(images[index], 0, 0, canvas.width, canvas.height);
}

gsap.to({ frame: 0 }, {
  frame: frameCount - 1,
  snap: 'frame',
  ease: 'none',
  scrollTrigger: {
    trigger: '.scroll-zoom-section',
    start: 'top top', end: 'bottom bottom', scrub: true,
  },
  onUpdate: function () { render(Math.round(this.targets()[0].frame)); },
});
```

■ **PRO TIP**

Generate the frame sequence with an AI video tool (Veo, Runway) or render it from Spline/Blender, then export every Nth frame as an image set.

Scroll Storytelling

Award sites use scroll as a narrative device — content is revealed, pinned, and transformed in sync with how far the user has scrolled, not just when it appears.

Three patterns every award site uses

Pinning — An element stays fixed on screen while the page scrolls past it, letting its internal animation play out fully before releasing.

Scrubbing — An animation's progress is tied directly to scroll position — scroll up, animation reverses. No fixed duration.

Staggered reveal — Groups of elements (text lines, cards, images) animate in with a slight delay between each, creating rhythm.

JS — Pinned section with scrubbed timeline (GSAP + Lenis)

```
import Lenis from 'lenis';
import { gsap } from 'gsap';
import { ScrollTrigger } from 'gsap/ScrollTrigger';
gsap.registerPlugin(ScrollTrigger);

const lenis = new Lenis();
lenis.on('scroll', ScrollTrigger.update);
gsap.ticker.add((time) => lenis.raf(time * 1000));

gsap.timeline({
  scrollTrigger: {
    trigger: '.pin-section',
    start: 'top top',
    end: '+=2000', // pin for 2000px of scroll
    pin: true,
    scrub: 1,
  }
})
.to('.title', { scale: 0.6, y: -200 })
.from('.cards', { opacity: 0, stagger: 0.15 });
```

■ PRO TIP

Prompt: 'Set up Lenis smooth scroll and a GSAP ScrollTrigger timeline that pins this section and staggers the .card elements in as the user scrolls.'

Cinematic Intro Loaders

The first 2-3 seconds set the entire tone. Award sites almost always use a preloader — not for actual loading time, but to control the first impression.

Progress counter

A number ticks 0 → 100% while assets load, then wipes away. Simple, communicates craft.

Logo reveal animation

Brand mark draws itself in (SVG path animation) before the wipe transition.

Color wipe transition

A solid color panel slides up/away to reveal the page — masks any layout shift.

Text scramble intro

Headline characters scramble/decode into place — popular on dev portfolios.

JS — Simple progress counter + wipe transition

```
const counter = document.querySelector('.loader-count');
const overlay = document.querySelector('.loader-overlay');
let progress = 0;

const interval = setInterval(() => {
  progress += Math.random() * 12;
  if (progress >= 100) {
    progress = 100;
    clearInterval(interval);
    gsap.to(overlay, {
      yPercent: -100, duration: 1, ease: 'power4.inOut',
      delay: 0.3,
    });
  }
  counter.textContent = Math.floor(progress) + '%';
}, 120);
```

■ PRO TIP

Keep loaders under 2.5 seconds even if assets load faster — fake the minimum duration so it never feels rushed or broken.

Color & Motion Direction

What actually separates an award site from a generic 3D demo isn't the tech — it's that every color, easing curve, and timing decision points the same direction.

The consistency checklist

- One accent color used for every interactive element (cursor, links, highlights) — never introduce a second accent.
- One easing curve reused across the whole site (e.g. `power3.inOut`) instead of mixing ease types per section.
- Motion speed scales with element size — small UI moves fast (200-300ms), full-page transitions move slow (800ms-1.2s).
- Background videos / 3D scenes stay in 2-3 brand colors max — desaturate or recolor stock-feeling renders.

GSAP — define one shared ease as a constant

```
// gsap-config.js – import this everywhere
export const EASE = 'power3.inOut';
export const DURATION = { fast: 0.3, base: 0.6, slow: 1.1 };

// usage
gsap.to('.card', { y: 0, duration: DURATION.base, ease: EASE });
```

■ PRO TIP

Prompt: 'Generate a dark color palette with one teal accent (#0D9488) suitable for a 3D product landing page, in OKLCH format.'

AI Shortcuts For Every Step

You don't need to hand-write every line above. Here's exactly what to ask an AI coding tool (Cursor, Claude Code, etc.) to get each piece built fast.

Stack setup	'Set up a Next.js project with GSAP, ScrollTrigger, and Lenis smooth scroll wired together correctly.'
3D scene	'Add a rotating Three.js icosahedron in the hero section with a teal MeshStandardMaterial and ambient + directional lighting.'
Image sequence	'Build a canvas-based scroll scrubber that plays a 120-frame image sequence from /public/frames as the user scrolls through this section, using GSAP ScrollTrigger.'
Pinning & reveal	'Pin the .hero section for 2000px of scroll and stagger-reveal the .feature-cards as it unpins.'
Loader	'Create a preloader with a 0-100% counter that fades to reveal the page after a minimum 2 second display.'
Performance pass	'Audit this component for scroll jank and add will-change, passive scroll listeners, and a mobile fallback for the Three.js scene.'

■ PRO TIP

Always specify the exact library names in your prompt (GSAP, Three.js, Lenis) — vague prompts like 'add cool animation' produce generic, inconsistent code.

Performance Reality Check

This is the part most tutorials skip — and the reason most 3D website clones feel laggy and amateur instead of premium. Fix these before calling it done.

Where 3D sites die on mobile

- Real-time Three.js scenes render at full desktop quality on phones — always detect device and lower pixel ratio / particle count on mobile.
- Large GLTF/GLB 3D model files load on every device — compress with gltf-pipeline or use Draco compression.
- Video backgrounds autoplay unmuted or without playsinline — causes mobile browsers to block playback entirely.

Where scroll animations get janky

- Animating top/left/width instead of transform/opacity forces layout recalculation every frame.
- Too many individual ScrollTrigger instances on one page — batch related animations into one timeline.
- Missing will-change: transform on heavily animated elements prevents GPU layer promotion.

Before you ship, check

- Lighthouse performance score on mobile, not just desktop.
- Core Web Vitals — especially LCP (aim under 2.5s) and CLS (aim near 0).
- Real device testing — an iPhone SE or mid-range Android, not just your dev machine.

@aiwithcasti

Follow for more breakdowns on how award sites are built.

Three.js · GSAP · Lenis
Built for real workflows